

DOCUMENTO 05 — Modelo de Dados

Projeto: Lindvior

Versão: 1.0

Status: Concluído

1. Objetivo

Este documento define o Modelo de Dados do Lindvior.

Seu objetivo é especificar como as informações produzidas pelo domínio da aplicação serão persistidas em banco de dados, estabelecendo a estrutura das tabelas, seus atributos, relacionamentos, restrições de integridade, estratégias de persistência e convenções de modelagem.

Enquanto:

- o Documento 01 define a visão geral do projeto;
- o Documento 02 define os requisitos funcionais;
- o Documento 03 define as regras de negócio;
- o Documento 04 define o modelo de domínio;

este documento define como essas informações serão representadas, armazenadas e relacionadas no banco de dados da aplicação.

Este documento serve como referência para a implementação da camada de persistência, definição das migrações, entidades JPA e estratégias de armazenamento de dados.

2. Escopo

Este documento contempla:

- modelo relacional do banco de dados;
- estrutura das tabelas;
- definição das colunas;
- tipos lógicos dos atributos;
- chaves primárias;
- chaves estrangeiras;

- relacionamentos;
- restrições de integridade;
- restrições de unicidade;
- estratégias de persistência;
- estratégias de atualização;
- estratégias de exclusão;
- índices utilizados pela aplicação;
- convenções de modelagem.

Não fazem parte deste documento:

- regras de negócio;
 - arquitetura da aplicação;
 - implementação utilizando JPA ou Hibernate;
 - detalhes específicos do PostgreSQL;
 - endpoints da API;
 - DTOs;
 - validações da camada de aplicação;
 - lógica da simulação.
-

3. Organização do Documento

Este documento está organizado nos seguintes capítulos.

4. Modelo Relacional

Apresenta a visão geral do banco de dados e o diagrama relacional das entidades persistentes.

5. Estrutura das Tabelas

Define detalhadamente cada tabela do banco de dados, suas colunas, tipos lógicos, restrições e observações de persistência.

6. Relacionamentos

Define todas as associações entre tabelas, cardinalidades, chaves estrangeiras e regras de integridade referencial.

7. Transações

Define as operações que deverão ser executadas de forma transacional para preservar a consistência do banco de dados.

8. Índices

Define os índices utilizados pela aplicação e a justificativa para sua existência.

9. Estratégias de Persistência

Define como inserções, atualizações, exclusões e preservação de histórico deverão ocorrer durante a operação da aplicação.

10. Convenções de Modelagem

Define os padrões utilizados para nomenclatura, organização e modelagem do banco de dados.

11. Evolução do Modelo

Define as diretrizes para evolução da estrutura persistente ao longo do projeto.

12. Considerações Gerais

Apresenta os princípios gerais do Modelo de Dados e sua integração com os demais documentos do Lindvior.

4. Modelo Relacional

4.1 Objetivo

Este capítulo apresenta a estrutura relacional do banco de dados do Lindvior.

Seu objetivo é demonstrar como as entidades persistentes se relacionam entre si, estabelecendo a organização geral da camada de persistência e preservando as regras de integridade definidas pelo Modelo de Domínio.

4.2 Visão Geral

O banco de dados do Lindvior foi projetado para representar diretamente as entidades do domínio descritas no Documento 04.

Cada tabela possui responsabilidade exclusiva sobre um conjunto específico de informações, evitando redundância de dados e preservando a consistência das relações existentes entre as entidades.

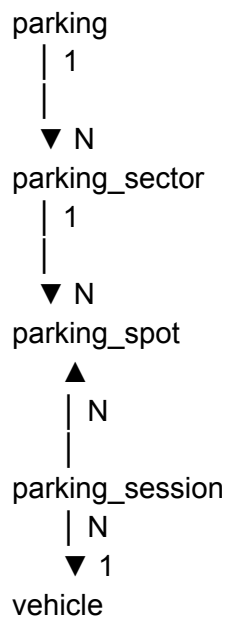
O modelo relacional é composto pelas seguintes tabelas:

- parking
- parking_sector

- parking_spot
 - vehicle
 - parking_session
 - user
-

4.3 Diagrama Relacional

A organização geral das tabelas pode ser representada da seguinte forma.



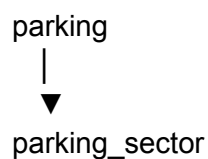
user

4.4 Organização do Modelo

O modelo relacional encontra-se dividido em três grupos principais.

Estrutura Física

Responsável por representar a estrutura permanente do estacionamento.



↓
parking_spot

Operação

Responsável por registrar toda utilização do estacionamento.

vehicle
↓
parking_session
↓
parking_spot

Administração

Responsável pelos usuários administrativos da aplicação.

user

4.5 Características do Modelo

O modelo relacional do Lindvior foi construído seguindo os seguintes princípios.

MD-RELATIONAL-001

Cada tabela representa exatamente uma entidade persistente do domínio.

MD-RELATIONAL-002

Relacionamentos são implementados exclusivamente por meio de chaves estrangeiras.

MD-RELATIONAL-003

Informações não deverão ser duplicadas entre tabelas.

MD-RELATIONAL-004

A estrutura deverá permitir a preservação completa do histórico operacional da aplicação.

MD-RELATIONAL-005

O modelo deverá permanecer normalizado, evitando redundâncias sempre que possível.

MD-RELATIONAL-006

Toda informação persistida deverá possuir um responsável único dentro do modelo de dados.

5. Estrutura das Tabelas

5.1 Objetivo

Este capítulo define detalhadamente a estrutura das tabelas que compõem o banco de dados do Lindvior.

Para cada tabela serão especificados:

- finalidade;
- estrutura;
- colunas;
- tipos lógicos;
- restrições;
- índices;
- observações de persistência.

As definições apresentadas neste capítulo deverão servir como referência para implementação das entidades persistentes e das migrações do banco de dados.

5.2 Tabela parking

Objetivo

Armazenar as informações gerais do estacionamento administrado pela aplicação.

O sistema deverá possuir exatamente um registro nesta tabela.

Estrutura

Coluna	Tipo Lógico	Obrigatório	P K	F K	Único	Observações
id	UUID	Sim	✓		✓	Identificador único do estacionamento.
name	Texto	Sim				Nome do estacionamento.
address	Texto	Sim				Endereço do estacionamento.
capacity	Inteiro	Sim				Deve ser maior que zero.
active	Booleano	Sim				Situação operacional.
created_at	Data/Hora	Sim				Data de criação do registro.

Restrições

- capacity deverá ser maior que zero.
- name não poderá ser vazio.
- address não poderá ser vazio.

Índices

- PK(id)
-

Observações

- Existirá apenas um registro durante toda a vida da aplicação.
 - A exclusão física deste registro não deverá ser permitida.
-

5.3 Tabela parking_sector

Objetivo

Armazenar os setores existentes no estacionamento.

Cada registro representa exatamente um setor.

Estrutura

Coluna	Tipo Lógico	Obrigatório	PK	FK	Único	Observações
id	UUID	Sim	✓		✓	Identificador único do setor.
parking_id	UUID	Sim		✓		Referencia parking.id .
name	Texto	Sim				Nome do setor.
type	Enum	Sim				Valores definidos por SectorType .

floor	Inteiro	Sim	Piso onde o setor está localizado.
active	Booleano	Sim	Situação operacional.
created_at	Data/Hora	Sim	Data de criação do registro.

Restrições

- parking_id deverá referenciar um estacionamento existente.
 - name deverá ser obrigatório.
 - type deverá utilizar a enumeração correspondente.
-

Índices

- PK(id)
 - IDX(parking_id)
-

Observações

- Um estacionamento poderá possuir diversos setores.
 - A desativação deverá preservar o histórico.
-

5.4 Tabela parking_spot

Objetivo

Armazenar todas as vagas pertencentes aos setores do estacionamento.

Cada registro representa exatamente uma vaga física.

Estrutura

Coluna	Tipo Lógico	Obrigatório	P K	F K	Único	Observações
id	UUID	Sim	✓		✓	Identificador único da vaga.
sector_id	UUID	Sim		✓		Referencia <code>parking_sector.id</code> .
code	Texto	Sim				Código identificador da vaga.
type	Enum	Sim				Valores definidos por <code>ParkingSpotType</code> .
status	Enum	Sim				Valores definidos por <code>ParkingSpotStatus</code> .
active	Booleano	Sim				Situação operacional.
created_at	Data/Hora	Sim				Data de criação do registro.

Restrições

- sector_id deverá referenciar um setor existente.

- code deverá ser único dentro do mesmo setor.
 - status deverá utilizar a enumeração correspondente.
-

Índices

- PK(id)
 - IDX(sector_id)
 - IDX(status)
 - IDX(active)
 - IDX(sector_id, code)
-

Observações

- A vaga poderá participar de inúmeras sessões durante sua existência.
- A exclusão física não deverá ser permitida.

5.5 Tabela vehicle

Objetivo

Armazenar os veículos pertencentes à população simulada.

Cada registro representa um veículo que poderá participar de diversas sessões de estacionamento ao longo de sua existência.

Estrutura

Coluna	Tipo Lógico	Obrigatório	PK	FK	Único	Observações
id	UUID	Sim	✓		✓	Identificador único do veículo.
license_plate	Texto	Sim			✓	Placa única do veículo.

type	Enum	Sim	Valores definidos por VehicleType .
color	Enum	Sim	Valores definidos por VehicleColor .
created_at	Data/Hora	Sim	Data de criação do registro.

Restrições

- license_plate deverá ser única.
 - type deverá utilizar a enumeração correspondente.
 - color deverá utilizar a enumeração correspondente.
-

Índices

- PK(id)
 - UK(license_plate)
-

Observações

- O veículo poderá participar de inúmeras sessões durante sua existência.
 - A exclusão física do veículo não deverá ser permitida para preservar o histórico das sessões.
-

5.6 Tabela parking_session

Objetivo

Armazenar cada utilização completa do estacionamento.

Cada registro representa uma sessão iniciada quando um veículo ocupa uma vaga e encerrada quando essa vaga é liberada.

Coluna	Tipo Lógico	Obrigatório	P K	F K	Único	Observações
id	UUID	Sim	✓		✓	Identificador único da sessão.
vehicle_id	UUID	Sim		✓		Referencia <code>vehicle.id</code> .
parking_spot_id	UUID	Sim		✓		Referencia <code>parking_spot.id</code> .
status	Enum	Sim				Valores definidos por <code>ParkingSessionStatus</code> .
entry_time	Data/Hora	Sim				Horário de ocupação da vaga.
exit_time	Data/Hora	Não				Preenchido ao finalizar a sessão.
created_at	Data/Hora	Sim				Data de criação do registro.

Restrições

- `vehicle_id` deverá referenciar um veículo existente.
- `parking_spot_id` deverá referenciar uma vaga existente.
- `exit_time` somente poderá possuir valor após o encerramento da sessão.

Índices

- PK(id)
- IDX(vehicle_id)
- IDX(parking_spot_id)
- IDX(status)
- IDX(status, vehicle_id)
- IDX(status, parking_spot_id)

Observações

- Uma sessão nunca poderá existir sem um veículo.
 - Uma sessão nunca poderá existir sem uma vaga.
 - Sessões finalizadas deverão permanecer armazenadas permanentemente.
-

5.7 Tabela user

Objetivo

Armazenar os usuários responsáveis pela administração da aplicação.

Estrutura

Coluna	Tipo Lógico	Obrigatório	PK	FK	Único	Observações
id	UUID	Sim	✓		✓	Identificador único do usuário.
email	Texto	Sim			✓	Utilizado para autenticação.
password	Texto	Sim				Armazena exclusivamente a senha criptografada.
role	Enum	Sim				Valores definidos por Role .
active	Booleano	Sim				Situação do usuário.

created_at	Data/Hora	Sim	Data de criação do registro.
------------	-----------	-----	------------------------------

updated_at	Data/Hora	Sim	Data da última atualização.
------------	-----------	-----	-----------------------------

Restrições

- email deverá ser único.
 - password deverá armazenar exclusivamente a senha criptografada.
 - role deverá utilizar a enumeração correspondente.
-

Índices

- PK(id)
 - UK(email)
 - IDX(active)
-

Observações

- Usuários desativados deverão permanecer armazenados.
 - A exclusão física não deverá ser permitida para preservar o histórico de auditoria.
-

5.8 Considerações sobre as Tabelas

MD-TAB-001

Toda tabela deverá possuir uma chave primária única e imutável.

MD-TAB-002

As colunas deverão armazenar apenas informações pertencentes à entidade que representam.

MD-TAB-003

Toda chave estrangeira deverá referenciar um registro existente.

MD-TAB-004

As restrições definidas para cada tabela deverão ser implementadas diretamente no banco de dados.

MD-TAB-005

A estrutura das tabelas deverá permanecer consistente com o Modelo de Domínio definido no Documento 04.

MD-TAB-006

Novas colunas somente poderão ser adicionadas quando representarem novas necessidades do domínio ou da persistência.

5.9 Constraints

Objetivo

Este capítulo consolida as principais restrições estruturais aplicadas ao banco de dados do Lindvior.

As constraints definidas neste capítulo deverão ser implementadas diretamente no banco de dados para garantir a integridade das informações persistidas.

Chaves Primárias

Tabela	Constraint
parking	PK(id)
parking_sector	PK(id)
parking_spot	PK(id)
vehicle	PK(id)
parking_session	PK(id)
user	PK(id)

Chaves Estrangeiras

Constraint	Referência
FK(parking_sector.parking_id)	parking.id
FK(parking_spot.sector_id)	parking_sector.id
FK(parking_session.vehicle_id)	vehicle.id

FK(parking_session.parking_spot_id) parking_spot.id

Constraints de Unicidade

Constraint

UK(vehicle.license_plate)

UK(user.email)

UK(parking_spot.sector_id, code)

Constraints CHECK

Constraint

CHECK(parking.capacity > 0)

Constraints NOT NULL

Todas as colunas marcadas como obrigatórias nas tabelas deste documento deverão possuir restrição **NOT NULL** no banco de dados.

6. Relacionamentos

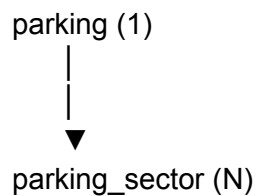
6.1 Objetivo

Este capítulo define como as tabelas do banco de dados do Lindvior se relacionam entre si.

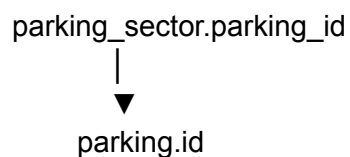
Seu objetivo é estabelecer as cardinalidades, chaves estrangeiras e regras de integridade referencial necessárias para preservar a consistência das informações persistidas.

6.2 parking → parking_sector

Cardinalidade



Chave Estrangeira



Comportamento da Chave Estrangeira

ON UPDATE RESTRICT

ON DELETE RESTRICT

Integridade

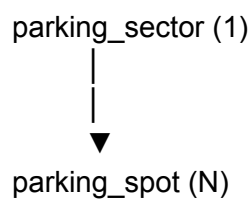
- Todo setor deverá pertencer a um estacionamento existente.
 - Não poderá existir setor sem estacionamento.
 - A exclusão do estacionamento deverá ser impedida enquanto existirem setores vinculados.
-

Tipo de Relacionamento

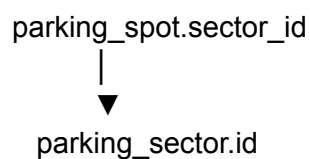
Um para Muitos (1:N)

6.3 parking_sector → parking_spot

Cardinalidade



Chave Estrangeira



Comportamento da Chave Estrangeira

ON UPDATE RESTRICT

ON DELETE RESTRICT

Integridade

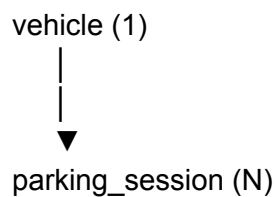
- Toda vaga deverá pertencer a um setor existente.
 - Não poderão existir vagas órfãs.
 - A remoção do setor deverá respeitar as regras de persistência definidas neste documento.
-

Tipo de Relacionamento

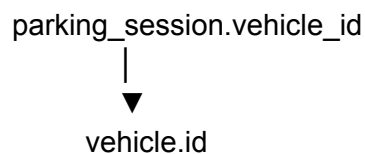
Um para Muitos (1:N)

6.4 vehicle → parking_session

Cardinalidade



Chave Estrangeira



Comportamento da Chave Estrangeira

ON UPDATE RESTRICT

ON DELETE RESTRICT

Integridade

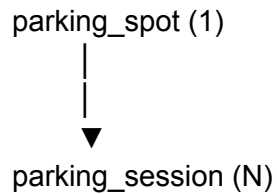
- Toda sessão deverá possuir um veículo válido.
 - Um veículo poderá possuir inúmeras sessões ao longo de sua existência.
 - Um veículo não poderá possuir duas sessões ativas simultaneamente, conforme definido pelas regras de negócio.
-

Tipo de Relacionamento

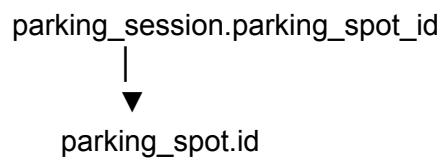
Um para Muitos (1:N)

6.5 parking_spot → parking_session

Cardinalidade



Chave Estrangeira



Comportamento da Chave Estrangeira

ON UPDATE RESTRICT

ON DELETE RESTRICT

Integridade

- Toda sessão deverá utilizar uma vaga existente.
 - Uma vaga poderá possuir inúmeras sessões históricas.
 - Uma vaga não poderá possuir mais de uma sessão ativa simultaneamente.
-

Tipo de Relacionamento

Um para Muitos (1:N)

6.6 Resumo dos Relacionamentos

Origem	Destino	Cardinalidade
parking	parking_sector	1:N
parking_sector	parking_spot	1:N
vehicle	parking_session	1:N
parking_spot	parking_session	1:N

6.7 Regras Gerais de Integridade Referencial

MD-FK-001

Toda chave estrangeira deverá referenciar um registro existente.

MD-FK-002

Nenhuma chave estrangeira poderá possuir referências órfãs.

MD-FK-003

As cardinalidades deverão refletir exatamente o Modelo de Domínio definido no Documento 04.

MD-FK-004

A remoção de registros não poderá produzir inconsistências entre tabelas relacionadas.

MD-FK-005

Toda associação entre tabelas deverá representar uma relação existente no domínio da aplicação.

MD-FK-006

A integridade referencial deverá ser garantida diretamente pelo banco de dados, independentemente da aplicação.

7. Transações

7.1 Objetivo

Este capítulo define as operações que deverão ser executadas de forma transacional durante a persistência dos dados do Lindvior.

Seu objetivo é garantir que operações compostas sejam executadas integralmente, preservando a consistência do banco de dados e evitando estados intermediários inválidos.

Sempre que uma operação modificar duas ou mais tabelas relacionadas, ela deverá ser executada dentro de uma única transação.

7.2 Criação de uma ParkingSession

Objetivo

Persistir a entrada de um veículo no estacionamento.

Operações

BEGIN TRANSACTION

1. Validar existência do Vehicle.
2. Validar existência da ParkingSpot.
3. Validar que a vaga está LIVRE.
4. Validar que o veículo não possui sessão ativa.

5. Criar ParkingSession.

6. Atualizar ParkingSpot para OCCUPIED.

COMMIT

Rollback

Caso qualquer etapa falhe, nenhuma alteração deverá permanecer persistida.

7.3 Encerramento de uma ParkingSession

Objetivo

Persistir a saída de um veículo do estacionamento.

Operações

BEGIN TRANSACTION

1. Localizar ParkingSession ativa.
2. Registrar exit_time.
3. Alterar status da ParkingSession.
4. Atualizar ParkingSpot para FREE.

COMMIT

Rollback

Caso qualquer etapa falhe, nenhuma alteração deverá ser persistida.

7.4 Criação de uma ParkingSpot

Objetivo

Persistir uma nova vaga pertencente a um setor.

Operações

BEGIN TRANSACTION

1. Validar existência do setor.
2. Validar unicidade do código dentro do setor.
3. Criar ParkingSpot.

COMMIT

Rollback

A falha em qualquer etapa deverá cancelar toda a operação.

7.5 Atualização de uma ParkingSpot

Objetivo

Persistir alterações cadastrais de uma vaga.

Operações

BEGIN TRANSACTION

1. Localizar a vaga.
2. Validar as alterações.
3. Atualizar os dados.

COMMIT

Rollback

Nenhuma alteração deverá ser aplicada parcialmente.

7.6 Criação de um Vehicle

Objetivo

Persistir um novo veículo da população simulada.

Operações

BEGIN TRANSACTION

1. Validar unicidade da placa.
2. Criar Vehicle.

COMMIT

Rollback

Caso ocorra qualquer falha, o veículo não deverá ser persistido.

7.7 Criação de um User

Objetivo

Persistir um novo usuário administrativo.

Operações

BEGIN TRANSACTION

1. Validar unicidade do e-mail.
2. Persistir senha criptografada.
3. Criar User.

COMMIT

Rollback

Nenhum usuário deverá ser criado parcialmente.

7.8 Atualização de um User

Objetivo

Persistir alterações cadastrais de um usuário.

Operações

BEGIN TRANSACTION

1. Localizar usuário.
2. Validar alterações.
3. Atualizar informações.
4. Atualizar updated_at.

COMMIT

Rollback

A atualização deverá ser totalmente revertida em caso de falha.

7.9 Princípios das Transações

MD-TX-001

Toda operação que modificar mais de uma tabela deverá ser executada dentro de uma única transação.

MD-TX-002

Nenhuma transação poderá ser parcialmente persistida.

MD-TX-003

Toda falha durante uma transação deverá provocar rollback completo das alterações realizadas.

MD-TX-004

A confirmação da transação somente poderá ocorrer após todas as validações e persistências serem concluídas com sucesso.

MD-TX-005

As transações deverão preservar todas as restrições de integridade definidas neste documento.

MD-TX-006

A consistência do banco de dados deverá ser garantida mesmo durante operações contínuas da simulação.

8. Índices

8.1 Objetivo

Este capítulo define a estratégia de indexação do banco de dados do Lindvior.

Seu objetivo é otimizar o desempenho das operações de consulta realizadas pela aplicação, reduzindo o custo de acesso aos dados sem comprometer desnecessariamente as operações de escrita.

Os índices definidos neste capítulo refletem os padrões de acesso previstos pelos requisitos funcionais e pelo motor da simulação.

8.2 Índices Primários

Toda tabela deverá possuir um índice associado à sua chave primária.

Tabela	Índice
parking	PK(id)
parking_sector	PK(id)
parking_spot	PK(id)
vehicle	PK(id)
parking_session	PK(id)
user	PK(id)

Finalidade

Permitir identificação rápida e única de qualquer registro persistido.

8.3 Índices de Unicidade

Os seguintes atributos deverão possuir índices únicos.

Tabela	Coluna	Justificativa
vehicle	license_plate	Impedir duplicidade de veículos.
user	email	Impedir duplicidade de usuários.

parking_spot	(sector_id, code)	Garantir unicidade do código da vaga dentro do setor.
--------------	----------------------	---

8.4 Índices de Relacionamento

As colunas utilizadas como chaves estrangeiras deverão possuir índices próprios.

Tabela	Coluna	Justificativa
parking_sector	parking_id	Consulta dos setores pertencentes ao estacionamento.
parking_spot	sector_id	Consulta das vagas pertencentes ao setor.
parking_session	vehicle_id	Consulta do histórico de utilização do veículo.
parking_session	parking_spot_id	Consulta do histórico de utilização da vaga.

8.5 Índices Operacionais

Além dos índices estruturais, algumas consultas realizadas continuamente pela aplicação justificam índices adicionais.

Tabela	Coluna	Justificativa
parking_spot	status	Localização de vagas livres durante a simulação.
parking_spot	active	Consulta de vagas ativas.
parking_sector	active	Consulta de setores ativos.
parking_session	status	Consulta de sessões ativas.
user	active	Consulta de usuários ativos.

8.6 Índices Compostos

Algumas consultas frequentes utilizam simultaneamente mais de um atributo.

Para esses casos deverão ser utilizados índices compostos.

Tabela	Índice	Justificativa
parking_spot	(status, active)	Localização rápida de vagas livres e ativas.
parking_session	(status, vehicle_id)	Verificação de sessão ativa de um veículo.
parking_session	(status, parking_spot_id)	Verificação da sessão ativa de uma vaga.

8.7 Estratégia de Indexação

A estratégia de indexação do Lindvior deverá seguir os seguintes princípios.

MD-IDX-001

Todo índice deverá possuir uma finalidade operacional claramente definida.

MD-IDX-002

Índices não deverão ser criados sem justificativa baseada no padrão de consultas da aplicação.

MD-IDX-003

Índices compostos deverão ser utilizados apenas quando houver benefício comprovado para consultas envolvendo múltiplas colunas.

MD-IDX-004

A criação de novos índices deverá considerar o impacto nas operações de inserção, atualização e remoção de registros.

MD-IDX-005

A estratégia de indexação poderá ser revisada futuramente com base no comportamento observado durante a execução da simulação.

MD-IDX-006

A definição dos índices deverá permanecer compatível com a evolução do Modelo de Dados e dos requisitos funcionais da aplicação.

9. Estratégias de Persistência

9.1 Objetivo

Este capítulo define como os dados do Lindvior deverão ser persistidos durante toda a operação da aplicação.

Seu objetivo é estabelecer diretrizes para criação, atualização, preservação e remoção das informações armazenadas, garantindo consistência, integridade e compatibilidade com as regras de negócio.

9.2 Estratégia de Inserção

Toda inserção deverá representar a criação de uma nova entidade do domínio.

Após persistido, um registro passará a fazer parte permanentemente da base de dados, salvo quando houver regra específica determinando sua remoção.

Regras

MD-PERSIST-001

Toda inserção deverá respeitar as restrições de integridade definidas neste documento.

MD-PERSIST-002

Nenhum registro poderá ser persistido com referências inválidas.

MD-PERSIST-003

A criação de registros deverá ocorrer apenas após todas as validações de negócio serem concluídas.

MD-PERSIST-004

Toda inserção deverá registrar automaticamente a data de criação do registro.

9.3 Estratégia de Atualização

As atualizações deverão modificar apenas os atributos permitidos para cada entidade.

A identidade do registro deverá permanecer inalterada durante toda sua existência.

Regras

MD-PERSIST-005

A chave primária nunca poderá ser alterada.

MD-PERSIST-006

Toda atualização deverá preservar a integridade referencial.

MD-PERSIST-007

Sempre que aplicável, a data da última atualização deverá ser registrada automaticamente.

MD-PERSIST-008

Atualizações não poderão produzir estados incompatíveis com o Modelo de Domínio.

9.4 Estratégia de Exclusão

A exclusão física de registros deverá ser evitada sempre que comprometer o histórico da aplicação.

Sempre que possível deverá ser utilizada desativação lógica.

Estratégia por Entidade

Entidade	Estratégia
Parking	Exclusão não permitida
ParkingSector	Desativação
ParkingSpot	Desativação
Vehicle	Exclusão não permitida
ParkingSession	Exclusão não permitida
User	Desativação

Regras

MD-PERSIST-009

Nenhuma exclusão poderá produzir registros órfãos.

MD-PERSIST-010

Entidades históricas deverão permanecer armazenadas permanentemente.

MD-PERSIST-011

A exclusão física somente poderá ocorrer quando prevista pelas regras da aplicação.

9.5 Estratégia de Histórico

O banco de dados deverá preservar as informações necessárias para reconstrução completa do histórico operacional da aplicação.

Regras

MD-PERSIST-012

Toda ParkingSession deverá permanecer armazenada após seu encerramento.

MD-PERSIST-013

O histórico de utilização dos veículos deverá permanecer preservado.

MD-PERSIST-014

Usuários desativados deverão permanecer armazenados para preservação das informações administrativas.

MD-PERSIST-015

As informações históricas deverão permanecer disponíveis para consultas e relatórios.

9.6 Estratégia para Dados Derivados

Algumas informações poderão ser obtidas a partir de outros atributos persistidos.

Sempre que possível, esses dados deverão ser calculados durante a utilização da aplicação, evitando redundância de armazenamento.

Regras

MD-PERSIST-016

A duração de uma sessão deverá ser calculada a partir dos horários de entrada e saída.

MD-PERSIST-017

Indicadores estatísticos deverão ser obtidos por consultas ao banco de dados sempre que possível.

MD-PERSIST-018

Informações derivadas não deverão ser persistidas quando puderem ser reconstruídas de forma confiável.

9.7 Estratégia Geral de Persistência

MD-PERSIST-019

O banco de dados deverá representar fielmente o estado atual do domínio.

MD-PERSIST-020

Toda operação de persistência deverá preservar a consistência das entidades relacionadas.

MD-PERSIST-021

As estratégias definidas neste capítulo deverão permanecer compatíveis com o Modelo de Domínio e com as Regras de Negócio do Lindvior.

MD-PERSIST-022

A evolução da estrutura de persistência não deverá comprometer os dados previamente armazenados.

MD-PERSIST-023

Toda alteração realizada pela simulação deverá ser persistida de forma consistente antes da continuação do fluxo operacional.

10. Convenções de Modelagem

10.1 Objetivo

Este capítulo define as convenções utilizadas na modelagem do banco de dados do Lindvior.

Seu objetivo é garantir padronização, consistência e facilidade de manutenção durante toda a evolução da estrutura de persistência da aplicação.

As convenções aqui estabelecidas deverão ser seguidas por todas as tabelas, colunas, restrições e índices do banco de dados.

10.2 Convenções de Nomenclatura

Tabelas

Os nomes das tabelas deverão:

- utilizar letras minúsculas;
- utilizar o padrão *snake_case*;
- permanecer no idioma inglês;
- representar diretamente uma entidade do domínio.

Exemplos

parking

parking_sector

parking_spot

parking_session

vehicle

user

Colunas

Os nomes das colunas deverão:

- utilizar o padrão *snake_case*;
- permanecer no idioma inglês;
- possuir significado claro;
- evitar abreviações desnecessárias;
- manter nomenclatura consistente em todo o banco.

Exemplos

created_at

updated_at

license_plate

parking_spot_id

entry_time

exit_time

vehicle_id

10.3 Identificadores

Todas as tabelas deverão possuir um identificador único.

Regras

MD-CONV-001

Toda tabela deverá possuir uma única chave primária.

MD-CONV-002

Os identificadores deverão utilizar UUID como tipo lógico.

MD-CONV-003

A chave primária não deverá possuir significado de negócio.

MD-CONV-004

Relacionamentos entre tabelas deverão utilizar exclusivamente chaves estrangeiras.

10.4 Tipos de Dados

Os tipos lógicos utilizados neste documento deverão manter correspondência direta com os tipos físicos definidos durante a implementação.

Tipos utilizados

Tipo Lógico	Finalidade
UUID	Identificadores únicos
Texto	Informações textuais
Inteiro	Valores numéricos inteiros
Booleano	Estados verdadeiro/falso
Enum	Valores previamente definidos
Data/Hora	Registro temporal
Tempo	Intervalos de duração

10.5 Convenções de Auditoria

Sempre que aplicável, as tabelas deverão registrar informações temporais que permitam acompanhar a evolução dos dados.

Regras

MD-CONV-005

Toda entidade persistente deverá possuir `created_at`.

MD-CONV-006

Entidades que permitirem atualização deverão possuir `updated_at`.

MD-CONV-007

Registros históricos não deverão perder informações após sua criação.

10.6 Evolução do Modelo

A estrutura do banco poderá evoluir durante o desenvolvimento do projeto, desde que preserve compatibilidade com os documentos oficiais.

Regras

MD-CONV-008

Novas tabelas somente poderão ser criadas quando representarem novos conceitos do domínio.

MD-CONV-009

Novas colunas deverão preservar compatibilidade com o Modelo de Domínio.

MD-CONV-010

Alterações estruturais deverão ser documentadas antes de sua implementação.

MD-CONV-011

Mudanças no modelo não deverão comprometer a integridade dos dados existentes.

10.7 Princípios das Convenções

MD-CONV-012

A modelagem deverá permanecer simples, consistente e de fácil manutenção.

MD-CONV-013

Toda estrutura persistente deverá refletir diretamente o Modelo de Domínio.

MD-CONV-014

As convenções estabelecidas neste capítulo deverão ser aplicadas uniformemente em todas as tabelas.

MD-CONV-015

Toda evolução do banco de dados deverá preservar a consistência do modelo de persistência.

11. Evolução do Modelo

11.1 Objetivo

Este capítulo estabelece as diretrizes para evolução do Modelo de Dados do Lindvior.

Seu objetivo é garantir que futuras alterações na estrutura de persistência ocorram de forma controlada, preservando a integridade dos dados, a compatibilidade com o domínio e a estabilidade da aplicação.

Toda evolução do modelo deverá ser planejada, documentada e compatível com os documentos oficiais do projeto.

11.2 Inclusão de Novas Tabelas

Novas tabelas somente poderão ser adicionadas quando representarem novos conceitos do domínio ou novas necessidades de persistência identificadas durante a evolução da aplicação.

Regras

MD-EVO-001

Toda nova tabela deverá possuir uma finalidade claramente definida.

MD-EVO-002

Nenhuma tabela poderá ser criada apenas para duplicação de informações já existentes.

MD-EVO-003

Toda nova tabela deverá possuir chave primária própria.

MD-EVO-004

Toda nova tabela deverá ser integrada ao modelo relacional por meio das chaves estrangeiras necessárias.

11.3 Inclusão de Novas Colunas

Novos atributos poderão ser adicionados às tabelas existentes quando representarem novas necessidades do domínio ou da persistência.

Regras

MD-EVO-005

Novas colunas deverão preservar compatibilidade com os registros existentes.

MD-EVO-006

Campos obrigatórios adicionados posteriormente deverão possuir estratégia de migração para os registros já persistidos.

MD-EVO-007

Toda nova coluna deverá possuir finalidade claramente definida.

11.4 Alteração da Estrutura

Mudanças estruturais deverão preservar a consistência do banco de dados e a compatibilidade com os documentos oficiais do Lindvior.

Regras

MD-EVO-008

Alterações estruturais deverão ser precedidas por atualização da documentação.

MD-EVO-009

O Modelo de Dados deverá permanecer consistente com o Modelo de Domínio.

MD-EVO-010

Mudanças na estrutura persistente não deverão violar as regras de negócio da aplicação.

11.5 Migração de Dados

Sempre que alterações estruturais exigirem transformação dos dados existentes, deverão ser utilizadas migrações controladas.

Regras

MD-EVO-011

Toda migração deverá preservar a integridade dos dados existentes.

MD-EVO-012

Migrações deverão ser executadas de forma reproduzível e versionada.

MD-EVO-013

A execução de migrações não deverá produzir perda de informações persistidas.

11.6 Compatibilidade

A evolução do banco de dados deverá preservar compatibilidade com a arquitetura geral da aplicação.

Regras

MD-EVO-014

Toda alteração deverá permanecer compatível com os Documentos 01, 02, 03 e 04.

MD-EVO-015

Mudanças na persistência deverão refletir corretamente o estado do domínio.

MD-EVO-016

A evolução do modelo deverá preservar a consistência das entidades e de seus relacionamentos.

11.7 Princípios da Evolução

MD-EVO-017

A evolução do Modelo de Dados deverá ocorrer de forma incremental e controlada.

MD-EVO-018

Toda alteração estrutural deverá ser documentada antes de sua implementação.

MD-EVO-019

A estrutura do banco deverá permanecer simples, consistente e de fácil manutenção.

MD-EVO-020

A evolução do modelo deverá priorizar a preservação da integridade e da compatibilidade dos dados existentes.

12. Considerações Gerais

12.1 Objetivo

Este capítulo estabelece os princípios gerais que deverão orientar a utilização e a evolução do Modelo de Dados do Lindvior.

As definições apresentadas neste documento representam a estrutura oficial de persistência da aplicação e deverão servir como referência para implementação, manutenção e futuras evoluções do banco de dados.

12.2 Princípios do Modelo de Dados

MD-GEN-001

O Modelo de Dados deverá representar fielmente o Modelo de Domínio definido no Documento 04.

MD-GEN-002

Toda informação persistida deverá possuir finalidade claramente definida dentro da aplicação.

MD-GEN-003

A estrutura do banco de dados deverá preservar a integridade, consistência e rastreabilidade das informações armazenadas.

MD-GEN-004

A modelagem deverá minimizar redundâncias, armazenando apenas os dados necessários para o funcionamento da aplicação.

MD-GEN-005

O banco de dados deverá suportar a operação contínua da simulação, preservando o histórico completo produzido pela aplicação.

12.3 Integração com os Demais Documentos

O Documento 05 deverá ser interpretado em conjunto com os demais documentos oficiais do Lindvior.

A relação entre eles pode ser resumida da seguinte forma:

- **Documento 01** — define a visão geral do projeto e seu escopo.
- **Documento 02** — define os requisitos funcionais.
- **Documento 03** — estabelece as regras de negócio.
- **Documento 04** — define o modelo de domínio.
- **Documento 05** — define a estrutura de persistência utilizada pela aplicação.

Os documentos são complementares e deverão permanecer consistentes entre si.

12.4 Utilização durante o Desenvolvimento

Durante a implementação do Lindvior, este documento deverá servir como referência para:

- modelagem do banco de dados;
- criação das migrações;
- implementação das entidades persistentes;
- configuração dos relacionamentos;
- definição das restrições de integridade;
- implementação da camada de persistência;
- definição da estratégia de indexação;
- validação da consistência do banco de dados.

Sempre que houver dúvidas sobre a estrutura de armazenamento da aplicação, este documento deverá ser considerado a principal referência.

12.5 Evolução do Modelo

A estrutura definida neste documento poderá evoluir durante o desenvolvimento do projeto, desde que toda alteração preserve:

- compatibilidade com o Modelo de Domínio;

- integridade dos dados já persistidos;
- consistência das regras de negócio;
- rastreabilidade da evolução da estrutura de persistência.

Toda alteração deverá ser previamente documentada antes de sua implementação.

12.6 Considerações Finais

O Modelo de Dados do Lindvior representa a especificação oficial da estrutura de persistência da aplicação.

Seu propósito é garantir que todas as informações produzidas pelo domínio sejam armazenadas de forma íntegra, consistente e compatível com as regras de negócio estabelecidas pelos documentos anteriores.

As definições apresentadas neste documento deverão servir como base para a implementação das entidades persistentes, migrações, relacionamentos e mecanismos de acesso a dados, reduzindo decisões de implementação e assegurando que a camada de persistência permaneça alinhada ao domínio da aplicação durante toda a evolução do projeto.